

# UDP 타임코드 동기화 프로토콜

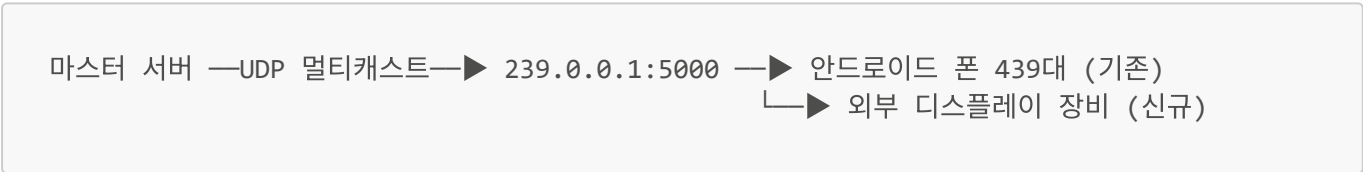
## MediaSphere 서버 UDP 타임코드 수신 연동

대상: 외부 디스플레이 장비(미디어 업체)

### 0. 확정된 값

| 항목       | 값                                              |
|----------|------------------------------------------------|
| 멀티캐스트 그룹 | 239.0.0.1                                      |
| 포트       | 5000 (UDP)                                     |
| 전송 주기    | 30fps (약 33ms 간격)                              |
| 서버 IP    | 192.168.8.10                                   |
| MQTT 브로커 | 192.168.8.10:1883 (재생 시작/정지 이벤트 수신용, 아래 4번 참고) |

### 1. 시스템 개요



- 서버는 재생 시작(PLAY) 이후 흐른 시간(**e**lapsed**M**s)을 30fps로 같은 스위치/네트워크에 있는 **모든** 장비에 동일하게 브로드캐스트. 폰 전용 프로토콜이 아니라, 같은 네트워크의 아무 장비나 패킷 수신가능.
- 외부 디스플레이 장비는 이 패킷을 받아서 **자기 영상의 재생 위치를 스스로 계산**하고, 자체 재생기(플레이어)의 위치를 그 값에 맞춰 보정. 서버가 영상을 스트리밍해주는 게 아니라, 각 장비가 이미 갖고 있는 동일한 영상 파일을 "같은 시각에 같은 위치"로 맞춰 재생하는 방식.

### 2. UDP 패킷 형식

- 전송 방식: UDP 멀티캐스트, payload는 UTF-8 JSON 문자열 (재전송/응답 없음 - 패킷 유실 시 그냥 다음 패킷을 기다리면 됨, 33ms 뒤 다음 패킷이 옴)

```
{
  "type": "TIMECODE",
  "masterMs": 1755300000123,
  "elapsedMs": 5231,
  "startAt": 1755299994892
}
```

| 필드   | 타입     | 설명            |
|------|--------|---------------|
| type | string | 항상 "TIMECODE" |

| 필드               | 타입      | 설명                                                                          |
|------------------|---------|-----------------------------------------------------------------------------|
| <b>masterMs</b>  | integer | 이 패킷을 보낸 시각의 서버 epoch ms(UTC 기준 1970-01-01부터의 ms). 네트워크 전송 지연 보정에 사용(3번 참고) |
| <b>elapsedMs</b> | integer | <b>PLAY가 눌린 시점부터 누적된 경과 시간(ms)</b> . 영상 길이로 나눈 나머지가 아니라 계속 커지는 값 — 4번 참고    |
| <b>startAt</b>   | integer | 이번 재생이 시작된 시각의 서버 epoch ms (참고용, 필수 아님)                                     |

### 3. 재생 위치 계산 방법

**elapsedMs**는 영상 길이를 모른 채 계속 누적되는 값. 서버는 각 장비가 재생하는 영상의 실제 길이(duration)를 전혀 알지 못하므로, **루프 위치 계산은 반드시 수신 측에서** 자기 영상의 실제 길이(**duration**, ms)로 나머지 연산을 해야 한다:

```
targetPositionMs = ((elapsedMs % duration) + duration) % duration
```

(+ duration) % duration은 혹시 모를 음수 방지용 — **elapsedMs**가 항상 0 이상이면 없어도 무방)

**주의: duration은 반드시 안드로이드 폰들이 재생 중인 영상과 정확히 같은 프레임 수/길이어야 함.** 미세하게라도 길이가 다르면(인코딩 과정에서 프레임이 반올림되는 경우 등) 시간이 지날수록 폰 벽면과 점점 어긋남.

#### 드리프트 보정 권장 방식

패킷마다 계산한 **targetPositionMs**와 자기 플레이어의 실제 재생 위치를 비교해서:

- 오차가 작으면(수십 ms 이내) 재생 속도를  $\pm 1\sim 5\%$  정도만 살짝 조절해서 서서히 맞춘다(급격한 점프 방지).
- 오차가 크면(수백 ms 이상) 그 위치로 즉시 seek한다.

안드로이드 쪽 구현(참고용 기준값 - 그대로 따를 필요는 없음):

| 오차 범위     | 처리             |
|-----------|----------------|
| 16ms 미만   | 무시(정상 속도 유지)   |
| 16~50ms   | 속도 $\pm 1.5\%$ |
| 50~100ms  | 속도 $\pm 3\%$   |
| 100~200ms | 속도 $\pm 5\%$   |
| 200ms 초과  | 즉시 seek        |

**주의 - 루프 경계:** **elapsedMs % duration** 특성상 영상이 한 바퀴 돌 때마다 **targetPositionMs**가 **duration** 근처  $\rightarrow 0$  근처로 순간 점프한다. 이 순간에 위 드리프트 비교를 그대로 하면 "오차가 영상 길이만큼 벌어졌다"고 오판해서 엉뚱하게 될 수 있다 — 직전 계산값 대비 **duration**의 50% 이상 점프했으면 "루프 경계를 막 넘은 순간"으로 보고 그 프레임의 보정은 건너뛰는 걸 권장.

### 4. 재생 시작/정지 이벤트 (권장 - MQTT)

UDP 타임코드는 PLAY 여부와 상관없이 항상 30fps로 브로드캐스트(정지 중엔 **elapsedMs**가 마지막 값에 멈춘 채로 계속 옴). 즉 UDP만으로는 "지금부터 재생을 시작하라"는 신호를 명확히 구분하기 어렵다 — 연속된 두 패킷의 **elapsedMs**가 그대로인지(정지) 늘어나는지(재생 중)로 유추는 가능하지만 번거로움.

**정확한 시작/정지 신호가 필요하면 MQTT를 함께 구독하는 걸 권장:**

브로커: mqtt://192.168.8.10:1883  
토픽: wall/control (인증 없음 - 폐쇄망 내부에서만 접근 가능한 걸 전제로 함)

| 이벤트   | 페이로드                                                |
|-------|-----------------------------------------------------|
| 재생 시작 | <b>{ "type": "PLAY", "startAt": 1755299994892 }</b> |
| 재생 정지 | <b>{ "type": "STOP", "elapsedMs": 5231 }</b>        |

PLAY/STOP 모두 retain 메시지라, 장비가 재부팅하거나 늦게 접속해도 재연결 즉시 "지금 재생 중인지 정지 중인지"를 바로 받을 수 있음. STOP의 **elapsedMs**도 3번과 같은 방식(% **duration**)으로 정지 위치를 계산해서 그 프레임에 멈추면 폰 벽면과 같은 화면으로 정지할 수 있음.

이 토픽에는 텍스트 스크롤/패턴, 컬러 오버레이 등 이 연동과 무관한 다른 명령도 함께 발행되니, **type** 필드로 PLAY/STOP만 걸러서 처리하면 된다.

## 5. 네트워크 연결 시 주의사항

- 5GHz 폐쇄망(안드로이드 폰들이 쓰는 것과 같은 네트워크)에 유선으로 연결하면 같은 멀티캐스트 도메인에 들어오는 구조 - 별도 라우팅 설정 없이 스위치 레벨에서 바로 수신 가능해야 함(라우터를 거쳐 서브넷이 분리되면 멀티캐스트 라우팅이 별도로 필요하니, 가능하면 같은 스위치/같은 서브넷에 물리적으로 연결할 것).
- 인증이 없는 열린 프로토콜 - 네트워크에 연결되는 모든 장비가 트래픽을 받을 수 있다는 전제로 설계됨.
- 5GHz Wi-Fi에도 연결 가능.